claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a163fa78b0b716a36.jsonl`
- SHA-256: `a6956729af7da0ec610172a960822244c6d998752b0da613ce7d18cffcfcc131`
- Source modified: `2026-06-22T09:49:37+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:21.902Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:
{
 "title": "Segmenter registry.get() returns CLASS not instance; correctly instantiated with (db, config)",
 "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/base.py:48-49, nightly_reelcount.py:335-345",
 "severity": "low",
 "detail": "Line 335 in nightly_reelcount.py: seg_cls = segmenter_registry.get(segmenter) returns the CLASS. Line 345 correctly instantiates: seg_cls(db, config).process_video(...). This matches the base.py contract (line 6-9 __init__ signature).",
 "fix": "No fix needed. The instantiation is correct."
}

2. assistant (2026-06-22T09:48:23.940Z)

I'll analyze the nightly reel-count engine-regression change by examining the files you've specified and verifying the specific finding about the segmenter registry.

3. user (2026-06-22T09:48:24.963Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
```

```
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8       owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10      What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12          Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13          **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15          ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25          python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28          python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
36      import datetime as _dt
37      import json
38      import os
39      import sys
40      from dataclasses import dataclass, field
41      from typing import Any, Dict, List, Optional, Tuple
42
43      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44      if REPO_ROOT not in sys.path:
45          sys.path.insert(0, REPO_ROOT)
46
47      # billing is pure (typing only) ? safe to import at module top.
48      from backend import billing  # noqa: E402
49
50      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54      #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
```

```
    QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
    DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the exact reel-count
    DEFAULT_FX_INR_PER_USD = 83.0
    #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-south1, ~mid-2026).
    DEFAULT_MACHINE_USD_PER_HR = 0.35


    @dataclass(frozen=True)
    class Tolerances:
        quality_tol: float = DEFAULT_QUALITY_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"quality_tol": self.quality_tol}


    # ---------------------------------------------------------------------- #
    # Pure cost math (wraps backend.billing ? no IO).
    # ---------------------------------------------------------------------- #
    def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
        """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing`` arithmetic).

        ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-uptime-seconds``;
        otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for boot/install)."""
        rate_per_sec = float(machine_usd_per_hr) / 3600.0
        usage = {billing.WALL_SECONDS: float(billed_seconds)}
        rates = {billing.WALL_SECONDS: rate_per_sec}
        usd, breakdown = billing.compute_cost_usd(usage, rates)
        return {
            "billed_seconds": round(float(billed_seconds), 1),
            "gpu_seconds": round(float(gpu_seconds), 1),
            "machine_usd_per_hr": float(machine_usd_per_hr),
            "usd": usd,
            "inr": billing.to_inr(usd, fx_inr_per_usd),
            "breakdown": breakdown,
        }


    # ---------------------------------------------------------------------- #
    # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
    # ---------------------------------------------------------------------- #
    def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                          cost: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable snapshot from per-video run results.

        Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None, wall_seconds}``.
        A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it, never hides it)."""
        per_video: Dict[str, Any] = {}
        for r in run_results:
            per_video[r["name"]] = {
                "reel_count": r.get("reel_count"),
                "ok": bool(r.get("ok", False)),
                "error": r.get("error"),
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}

```

```
113
114     @dataclass
115     class Finding:
116         video: str
117         metric: str                 # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
118         kind: str                   # "regression" | "improvement" | "stale"
119         baseline: Optional[float] = None
120         current: Optional[float] = None
121         detail: str = ""
122
123         def message(self) -> str:
124             if self.kind == "stale":
125                 return f"[STALE] {self.metric}: {self.detail}"
126             if self.metric == "error":
127                 return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128             b = "?" if self.baseline is None else f"{self.baseline:g}"
129             c = "?" if self.current is None else f"{self.current:g}"
130             tag = "REGRESSION" if self.kind == "regression" else "improvement"
131             return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134     @dataclass
135     class NightlyResult:
136         findings: List[Finding] = field(default_factory=list)
137         added: List[str] = field(default_factory=list)
138         removed: List[str] = field(default_factory=list)
139
140         @property
141         def regressions(self) -> List[Finding]:
142             return [f for f in self.findings if f.kind == "regression"]
143
144         @property
145         def improvements(self) -> List[Finding]:
146             return [f for f in self.findings if f.kind == "improvement"]
147
148         @property
149         def stale(self) -> List[Finding]:
150             return [f for f in self.findings if f.kind == "stale"]
151
152         def overall_status(self) -> str:
153             if self.regressions:
154                 return "REGRESSION"
155             if self.stale:
156                 return "STALE"
157             return "PASS"
158
159         def exit_code(self) -> int:
160             if self.regressions:
161                 return 1
162             if self.stale:
163                 return 4
164             return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173           quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
```

```
        """
        res = NightlyResult()
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count
produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
```

```
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ------------------------------------------------------------------------ #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
        # CSV: header start,end (or first two numeric columns)
        import csv
        out: List[Tuple[float, float]] = []
        with open(local, encoding="utf-8") as fh:
            for row in csv.reader(fh):
                try:
                    out.append((float(row[0]), float(row[1])))
```

```
                except (ValueError, IndexError):
                    continue
        return out


    def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
        """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
        codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
        optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
        quality scoring gracefully (skipped) without ever miscounting reels."""
        out: List[Tuple[float, float]] = []
        for s in segs:
            start = s.get("start_time", s.get("start"))
            end = s.get("end_time", s.get("end"))
            if start is not None and end is not None:
                out.append((float(start), float(end)))
        return out


    def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                      rerun_on_mismatch: bool = True,
                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
        """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.

        The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
        database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
        reproduce, a real change will."""
        import random
        import time

        from backend.eval import rally_seg_eval
        from backend.infrastructure.database import Database
        from backend.pipeline.segmenters import segmenter_registry
        from backend.results import Failure
        from backend.storage import fetch
        from backend.utils.hashing import compute_video_id

        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
```

```
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
```

```
'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count + quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
                            "defaults to the in-process pipeline wall time (a lower bound)")
        p.add_argument("--no-rerun-guard", action="store_true",
                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                       help="freeze the current reel counts + quality as the new baseline instead of checking")
```

```
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
                log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
            return 0

        if baseline is None:
            log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
            return 3

        result = compare(baseline, snapshot, tols)
        report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

```

```
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

4. user (2026-06-22T09:48:25.266Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
```

```
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

5. user (2026-06-22T09:48:26.443Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6       never mask a regression).
7
8       Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
```

```
10        ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12        ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
13                      (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                       lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16      Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17      and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                      export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19      """
20      from __future__ import annotations
21
22      import logging
23      import os
24      from email.message import EmailMessage
25      from typing import Optional
26
27      log = logging.getLogger("nightly_email")
28
29      DEFAULT_TO = "avi.dullu@gmail.com"
30      DEFAULT_HOST = "smtp.gmail.com"
31      DEFAULT_PORT = 587
32
33
34      def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35          """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36          msg = EmailMessage()
37          msg["Subject"] = subject
38          msg["From"] = from_addr
39          msg["To"] = to_addr
40          msg.set_content(body_md)
41          return msg
42
43
44      def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45          """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46          env = env if env is not None else os.environ
47          direct = env.get("NIGHTLY_SMTP_PASS")
48          if direct:
49              return direct
50          secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51          if not secret_ref:
52              return None
53          # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54          try:
55              from google.cloud import secretmanager  # type: ignore
56              client = secretmanager.SecretManagerServiceClient()
57              return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58          except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59              pass
60          try:
61              import subprocess
62              name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63              out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
```

```
secret", name],
                                    capture_output=True, text=True, timeout=30)
            return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
        except Exception as e:  # noqa: BLE001
            log.warning("secret-manager resolution failed: %s", e)
            return None


    def send_report(subject: str, body_md: str, to: Optional[str] = None,
                    env: Optional[dict] = None) -> bool:
        """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
        env = env if env is not None else os.environ
        to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
        host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
        port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
        user = env.get("NIGHTLY_SMTP_USER") or to_addr
        password = resolve_secret(env)
        if not password:
            log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
            return False

        import smtplib
        msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
        with smtplib.SMTP(host, port, timeout=60) as s:
            s.starttls()
            s.login(user, password)
            s.send_message(msg)
        log.info("emailed report to %s", to_addr)
        return True

```

6. user (2026-06-22T09:48:26.907Z)

```
#!/usr/bin/env bash
# Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
# (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
# an interactive session). Reads its config from instance metadata (set by launch.sh).
#
# Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
#        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
#        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
#
# Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
#   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
set -uo pipefail
exec > >(tee -a /var/log/nightly-regression.log) 2>&1
echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
BOOT_EPOCH=$(date +%s)

md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
NAME=$(md instance/name)
ZONE=$(md instance/zone | awk -F/ '{print $NF}')
REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
BUCKET=$(md instance/attributes/gcs-bucket || true)
```

```
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
```

```
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

7. user (2026-06-22T09:48:27.539Z)

```
1       #!/usr/bin/env bash
2       # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
VM runs
3       # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
stages the repo
4       # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a
cron on any
5       # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
checkout.
6       #
7       #   bash deploy/gcp/nightly_regression/launch.sh                # L4, stages HEAD,
kicks the VM
8       #   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
9       #
10      # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
? the VM-side
11      # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
<name> --zone=<z>.
12      set -uo pipefail
13
14      PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
15      BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
16      SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
17      GPU="${RALLY_GPU:-l4}"
18      NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
19      REF="${RALLY_REF:-HEAD}"
20      WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
21      REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
22      DO_EMAIL="${RALLY_EMAIL:-true}"
23      SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
pass/versions/latest}"
24      SMTP_USER="${NIGHTLY_SMTP_USER:-}"
25      EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
26      HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
27      STARTUP="$HERE/run_on_vm.sh"
```

```
28      export CLOUDSDK_CORE_DISABLE_PROMPTS=1
29
30      ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
31      if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
32
33      # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
bucket).
34      SHA="$(git rev-parse "$REF")"
35      TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36      echo "== staging repo $SHA -> $TGZ_URI =="
37      TMP_TGZ="$(mktemp --suffix=.tgz)"
38      git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39      gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40      rm -f "$TMP_TGZ"
41
42      # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43      ERR="$(mktemp)"
44      for Z in $ZONES; do
45        echo "== creating $NAME ($GPU) in $Z =="
46        if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55          echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and
self-delete."
56          echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57          echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58          rm -f "$ERR"; exit 0
59        fi
60        if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in
$Z ? next zone)"; continue; fi
61        echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62      done
63      echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64      rm -f "$ERR"; exit 1
65
```

8. user (2026-06-22T09:48:28.328Z)

```
1       #!/usr/bin/env bash
2       # Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
3       #   Cloud Scheduler (cron)  ?  Cloud Run JOB (the launcher image)  ?  launch.sh
4       #     ?  ephemeral GPU VM (regression + email + SELF-DELETE).
5       # The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU
VM is ephemeral).
6       #
7       # ? OWNER-RUN, ONCE, after a successful manual `bash
deploy/gcp/nightly_regression/launch.sh` dry-run.
8       # NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips
+ weights in
9       # gs://<bucket>, and the SMTP secret (see README "Enable").
10      #
```

```
#   bash deploy/gcp/nightly_regression/register_scheduler.sh          # build + deploy + schedule (03:00 IST)
#   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
set -euo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
REGION="${RALLY_RUN_REGION:-asia-south1}"
JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
CRON="${RALLY_CRON:-0 3 * * *}"          # 03:00 daily
TZ="${RALLY_TZ:-Asia/Kolkata}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-launcher:latest}"
REPO_ROOT="$(git rev-parse --show-toplevel)"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

echo "== enable APIs (run, scheduler, build, artifact registry) =="
gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"

echo "== ensure the Artifact Registry repo 'rally' exists =="
gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 || \
  gcloud artifacts repositories create rally --repository-format=docker --location="$REGION" --project="$PROJECT"

echo "== build + push the launcher image from the repo root =="
( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
    --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-corpus}/cloudbuild" \
    -f deploy/gcp/nightly_regression/Dockerfile.launcher . )

echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU VM) =="
gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
  --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
  --set-env-vars="RALLY_GPU=${RALLY_GPU:-l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"

echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
RUN_URI="https://${REGION}-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 && \
  gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT" --quiet || true
gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
  --schedule="$CRON" --time-zone="$TZ" \
  --uri="$RUN_URI" --http-method=POST \
  --oauth-service-account-email="$SA"

cat <<EOF

? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
   IAM the launcher SA needs (grant once if missing):
     roles/compute.instanceAdmin.v1  (create/delete the GPU VM)
     roles/iam.serviceAccountUser    (attach $SA to the VM)
     roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
   Trigger now to test:  gcloud run jobs execute $JOB --region=$REGION --project=$PROJECT
   Remove:               gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud run jobs delete $JOB --region=$REGION
EOF

```

9. assistant (2026-06-22T09:48:32.020Z)

Now let me read the remaining files and then examine the specific code around the finding:

10. user (2026-06-22T09:48:32.691Z)

```
1       # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2       # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3       # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4       FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
5       WORKDIR /app
6       # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7       COPY . /app
8       ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9
```

11. user (2026-06-22T09:48:33.892Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

12. user (2026-06-22T09:48:35.065Z)

```
Found 17 files
scripts\nightly_reelcount.py
tools\generate_highlights.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_stub_runner.py
tests\test_base.py
scripts\run_process_and_stitch.py
scripts\run_phase3_eval.py
backend\worker\__main__.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\base.py
backend\main.py
```

13. assistant (2026-06-22T09:48:38.037Z)

Let me search for more specific context around line 335-345 in nightly_reelcount.py and examine how segmenter_registry.get() is used:

14. user (2026-06-22T09:48:39.634Z)

```
330         except Exception as e:  # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
```

15. user (2026-06-22T09:48:41.212Z)

```
322:    from backend.pipeline.segmenters import segmenter_registry
335:    seg_cls = segmenter_registry.get(segmenter)
345:        result = seg_cls(db, config).process_video(video_id, local)
```

16. assistant (2026-06-22T09:48:43.441Z)

Perfect. Now let me verify the segmenter_registry implementation by examining the base.py file more closely, particularly the `get()` method and registry structure:

17. user (2026-06-22T09:48:44.871Z)

```
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
```

18. user (2026-06-22T09:48:46.119Z)

```
./backend/pipeline/segmenters/base.py
./backend/pipeline/segmenters/gemini.py
./backend/pipeline/segmenters/motion.py
./backend/pipeline/segmenters/trajectory_hybrid.py
./backend/pipeline/segmenters/yolo_hybrid.py
./backend/pipeline/segmenters/__init__.py
./tests/test_base.py
```

19. assistant (2026-06-22T09:48:48.118Z)

Let me also check how the segmenters are actually used in other parts of the codebase to confirm the pattern:

20. user (2026-06-22T09:48:49.533Z)

```
No files found
```

21. user (2026-06-22T09:48:50.199Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import shutil
7       import socket
8       import threading
9       import uuid
10      import webbrowser
11      from importlib.metadata import PackageNotFoundError, version as _pkg_version
12      from urllib.parse import quote
13      import uvicorn
14      from dotenv import load_dotenv
15
16      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
```

```
19      # stdlib-only import ? keeps this pre-pydantic line light.
20      from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22      load_dotenv(env_file_to_load())
23
24      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25      logging.basicConfig(
26          level=logging.INFO,
27          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28          datefmt='%H:%M:%S'
29      )
30      logger = logging.getLogger(__name__)
31      from fastapi import FastAPI, HTTPException, Request
32      from fastapi.middleware.cors import CORSMiddleware
33      from starlette.middleware.trustedhost import TrustedHostMiddleware
34      from pydantic import BaseModel
35      from typing import List, Dict, Any, Optional
36
37      from fastapi.staticfiles import StaticFiles
38      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39      from backend.pipeline.validators.base import registry
40      from backend.infrastructure.database import Database
41      from backend.pipeline.segmenters.base import segmenter_registry
42      from backend.pipeline.stitcher.compiler import VideoCompiler
43      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
45      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
46      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
47      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
48      from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
49      from backend.config import (  # new typed config
50          load_config as load_app_config,
51          write_light_default_config,  # P2b: batteries-included first-run config seeding
52          AppConfig,
53          StitchingConfig,
54          enforce_startup_checks,
55          StartupCheckError,
56      )
57      from backend.results import Failure  # standardized error/result contract
58      from backend.reporting import emit_indexer_report
59      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
60      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
61      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
62
63      app = FastAPI(title="Sports Highlight Indexer API")
64
65
66      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
67          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
68          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
69          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
70          src = os.environ if env is None else env
71          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
72          origins = [o.strip() for o in raw.split(",") if o.strip()]
73          return origins or ["*"]
74
```

```
75
76      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
77      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
78      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
79      # lock it to its origin; the middleware is fixed at startup.
80      app.add_middleware(
81          CORSMiddleware,
82          allow_origins=_cors_origins_from_env(),
83          allow_credentials=False,
84          allow_methods=["*"],
85          allow_headers=["*"],
86      )
87
88
89      def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
90          """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
91          web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
92          the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
93          Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
94          Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
95          ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
96          deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
97          src = os.environ if env is None else env
98          raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
99          hosts = [h.strip() for h in raw.split(",") if h.strip()]
100         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
```

22. assistant (2026-06-22T09:48:52.522Z)

Let me look for actual usages of segmenter_registry.get() in the main backend:

23. user (2026-06-22T09:48:53.967Z)

```
backend/main.py-
backend/main.py:        segmenter_cls = segmenter_registry.get(seg_name)
backend/main.py-        if not segmenter_cls:
backend/main.py-            # Submit-time validation already 400s unknown names; this is the
defensive
backend/main.py-            # in-worker path (e.g. config default pointing at an unregistered
segmenter).
--
backend/main.py-        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
backend/main.py:    if req.segmenter and not segmenter_registry.get(req.segmenter):
backend/main.py-        available = list(segmenter_registry.list_available().keys())
backend/main.py-        raise HTTPException(
backend/main.py-            status_code=400,
--
backend/main.py-            # Index
backend/main.py:            segmenter_cls = segmenter_registry.get(seg_name)
backend/main.py-            if not segmenter_cls:
backend/main.py-                print(f"[FAIL] Segmenter '{seg_name}' not found.")
backend/main.py-                return
--
```

```
backend/worker/__main__.py-        seg_name = args.segmenter or
config.indexing.default_segmenter
backend/worker/__main__.py:        segmenter_cls = segmenter_registry.get(seg_name)
backend/worker/__main__.py-        if not segmenter_cls:
backend/worker/__main__.py-            print(f"[worker] unknown segmenter: {seg_name!r} "
backend/worker/__main__.py-                  f"(have
{list(segmenter_registry.list_available())})")
--
backend/worker/__main__.py-
backend/worker/__main__.py:    seg_cls = segmenter_registry.get(args.segmenter)
backend/worker/__main__.py-    if not seg_cls:
backend/worker/__main__.py-        print(f"[worker] unknown segmenter: {args.segmenter!r} "
backend/worker/__main__.py-              f"(have {list(segmenter_registry.list_available())})")
--
scratch/test_ai_segmenter.py-    print("Running 'gemini' segmenter (calling Gemini 2.5 Flash
Multimodal API)...")
scratch/test_ai_segmenter.py:    gemini_cls = segmenter_registry.get("gemini")
scratch/test_ai_segmenter.py-    if gemini_cls:
scratch/test_ai_segmenter.py-        segmenter = gemini_cls(db, config)
scratch/test_ai_segmenter.py-        rallies = segmenter.process_video(video_id, video_path)
--
scripts/nightly_reelcount.py-    gts = _load_gts(video.get("labels_uri"))
scripts/nightly_reelcount.py:    seg_cls = segmenter_registry.get(segmenter)
scripts/nightly_reelcount.py-    if not seg_cls:
scripts/nightly_reelcount.py-        return {"name": name, "reel_count": None, "ok": False,
scripts/nightly_reelcount.py-                "error": f"segmenter '{segmenter}' not registered",
"quality": None, "wall_seconds": 0.0}
--
scripts/run_phase3_eval.py-    if not args.score_only:
scripts/run_phase3_eval.py:        seg_cls = segmenter_registry.get(args.segmenter)
scripts/run_phase3_eval.py-        if not seg_cls:
scripts/run_phase3_eval.py-            print(f"ERROR: segmenter '{args.segmenter}' not found. "
scripts/run_phase3_eval.py-                  f"Available:
{list(segmenter_registry.list_available().keys())}", file=sys.stderr)
--
scripts/run_process_and_stitch.py-        seg_name = cfg.indexing.default_segmenter
scripts/run_process_and_stitch.py:        segmenter_cls = segmenter_registry.get(seg_name)
scripts/run_process_and_stitch.py-        if not segmenter_cls:
scripts/run_process_and_stitch.py-            print(f"[FAIL] Segmenter '{seg_name}' not found.")
scripts/run_process_and_stitch.py-            return
--
tools/generate_highlights.py-    db = Database(db_path)
tools/generate_highlights.py:    seg_cls = segmenter_registry.get(segmenter)
tools/generate_highlights.py-    if seg_cls is None:
tools/generate_highlights.py-        raise SystemExit(f"unknown segmenter '{segmenter}';
available: {segmenter_registry.list_available()}")
tools/generate_highlights.py-
```

24. assistant (2026-06-22T09:48:56.517Z)

Excellent. Now let me look at one concrete example to verify the pattern ? let me check the backend/worker/__main__.py to see how it instantiates the segmenter:

25. user (2026-06-22T09:48:58.151Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
```

```
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16      from __future__ import annotations
17
18      import argparse
19      import csv
20      import json
21      import logging
22      import os
23      import sys
24      import tempfile
25      from typing import Any, List
26
27      from backend import storage
28      from backend.compute import ComputeJobSpec, get_compute_backend
29      from backend.config import (
30          StartupCheckError,
31          enforce_startup_checks,
32          load_config,
33          probe_cuda_available,
34      )
35      from backend.infrastructure.database import Database
36      from backend.pipeline.segmenters.base import segmenter_registry
37      from backend.pipeline.validators.base import registry as validator_registry
38      from backend.reporting import emit_indexer_report
39      from backend.results import Failure
40      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
41      from backend.utils.hashing import compute_video_id
42
43      logger = logging.getLogger("backend.worker")
44
45
46      def _coerce(v: str) -> Any:
47          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
48          low = v.lower()
49          if low in ("true", "false"):
50              return low == "true"
51          for cast in (int, float):
52              try:
53                  return cast(v)
54              except ValueError:
55                  pass
56          return v
57
58
59      def _apply_overrides(config: Any, sets: List[str]) -> None:
60          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
61          for kv in sets or []:
62              key, sep, val = kv.partition("=")
63              if not sep:
64                  raise ValueError(f"--set expects k=v, got {kv!r}")
65              parts = key.split(".")
66              obj = config
67              for p in parts[:-1]:
68                  obj = getattr(obj, p)
69              setattr(obj, parts[-1], _coerce(val))
```

```
70
71
72      def _write_intervals_csv(segments: List[dict], path: str) -> None:
73          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
74          artifact (same schema as the rally-annotator labels)."""
75          with open(path, "w", newline="") as f:
76              w = csv.writer(f)
77              w.writerow(["start", "end", "shot_count", "ending_reason"])
78              for s in segments:
79                  w.writerow([
80                      f'{float(s.get("start_time", 0.0)):.3f}',
81                      f'{float(s.get("end_time", 0.0)):.3f}',
82                      s.get("shot_count", ""),
83                      s.get("ending_reason", s.get("shot_count_confidence", "")),
84                  ])
85
86
87      def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
88          with open(path, "w") as f:
89              json.dump({
90                  "video_id": video_id,
91                  "status": status,
92                  "segment_count": len(segments),
93                  "segments": [{"start": float(s.get("start_time", 0.0)),
94                                "end": float(s.get("end_time", 0.0))} for s in segments],
95              }, f, indent=2)
96
97
98      def _run_startup_checks(config: Any) -> bool:
99          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
100         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
101         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
102         (returns True, ZERO work) when no deployment.profile is selected.
103
104         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
105         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
106         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
107         no-op of work, not just of output."""
108         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
109         cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
110         try:
111             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
112             return True
113         except StartupCheckError:
114             return False  # already logged at ERROR by enforce_startup_checks
115
116
117     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
118         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
119         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
120         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
121
122         A remote job that never writes a completion marker (a failed/hung container) makes
```

```
the bucket
123         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
124         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
125         from backend.infrastructure.execution_policy import PolicyTimeout
126
127         spec = ComputeJobSpec(
128             video_uri=args.video_uri, out_uri=args.out_uri,
129             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
130         )
131         try:
132             result = compute.run_infer(spec)
133         except PolicyTimeout as e:
134             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
135             return 4
136         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
137                     result.video_id, result.returncode, result.outputs_uri)
138         return result.returncode
139
140
141     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
142                            status: str, storage_cfg: dict, scratch: str) -> None:
143         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
144         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
145         try:
146             marker = {"video_id": video_id, "returncode": returncode,
147                       "segment_count": segment_count, "status": status}
148             local = os.path.join(scratch, "_done.json")
149             with open(local, "w", encoding="utf-8") as f:
150                 json.dump(marker, f)
```

26. assistant (2026-06-22T09:49:00.876Z)

Let me continue reading the worker to see how segmenter_cls is instantiated:

27. user (2026-06-22T09:49:02.614Z)

```
    seg_cls = segmenter_registry.get(args.segmenter)
    if not seg_cls:
        print(f"[worker] unknown segmenter: {args.segmenter!r} "
              f"(have {list(segmenter_registry.list_available())})")
        return None
    seg = seg_cls(None, config)
```

28. assistant (2026-06-22T09:49:07.497Z)

Perfect! Let me also check the tools/generate_highlights.py example to see the full pattern:

29. user (2026-06-22T09:49:09.653Z)

```
    seg_cls = segmenter_registry.get(segmenter)
    if seg_cls is None:
        raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

    # Combined batch log (local; published at the end).
    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")
    if not logging.getLogger().handlers:
        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
```

```
        for h in logging.getLogger().handlers:
```

30. assistant (2026-06-22T09:49:12.098Z)

Let me look at the main backend to see the full context of the instantiation:

31. user (2026-06-22T09:49:13.164Z)

```
        # 2. Run segmenter & indexer
        logger.info("[API] Video passed checks. Segmenting and indexing...")
        db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r in
val_results])

        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            # Submit-time validation already 400s unknown names; this is the defensive
            # in-worker path (e.g. config default pointing at an unregistered segmenter).
            available = list(segmenter_registry.list_available().keys())
            response = {
                "video_id": video_id,
                "status": "failed",
                "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                "results": [r.model_dump() for r in val_results],
                "play_segments": [],
--
            if failures:
                print("[FAIL] Video failed checks. Indexing halted.")
                return

            # Index
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return

            print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
            segmenter_actual = seg_name
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
            segmentation_ran = True
```

32. assistant (2026-06-22T09:49:15.510Z)

Perfect! Now I have all the evidence I need. Let me verify one more time by examining the actual instantiation in nightly_reelcount.py around line 345:

33. user (2026-06-22T09:49:16.636Z)

```
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
```

```
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
```

34. assistant (2026-06-22T09:49:18.717Z)

Now I have complete evidence. Let me verify the BasePlaySegmenter.__init__ signature one final time:

35. user (2026-06-22T09:49:19.729Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
```

36. assistant (2026-06-22T09:49:29.215Z)

Perfect! I now have all the concrete evidence needed to verify the finding. Let me summarize:

Evidence Analysis

VERIFIED: The finding is CORRECT ? this is NOT a defect; the code is working as designed.

Code Evidence Chain:

1. **Line 48-49 in base.py** ? `segmenter_registry.get()` returns the CLASS (type), not an instance:
   ```python
   def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
       return self._segmenters.get(name)
   ```

2. **Line 335 in nightly_reelcount.py** ? `seg_cls` holds the CLASS:
   ```python
   seg_cls = segmenter_registry.get(segmenter)  # Returns Type[BasePlaySegmenter]
   ```

3. **Line 336-338 ? proper null check before instantiation:**
   ```python
   if not seg_cls:
       return {"name": name, "reel_count": None, "ok": False, ...}
   ```

4. **Line 345 in nightly_reelcount.py** ? seg_cls is CORRECTLY instantiated with `(db, config)`:
   ```python
   result = seg_cls(db, config).process_video(video_id, local)
   ```
   This matches the `__init__` signature on **line 7 of base.py**:
   ```python
   def __init__(self, db: Database, config: Dict[str, Any]):
   ```

5. **Corroborated pattern across the codebase:**

- `backend/main.py`: `segmenter = segmenter_cls(db_instance, global_config)`
- `backend/worker/__main__.py`: `seg = seg_cls(None, config)`
- `tools/generate_highlights.py`: same pattern

Conclusion:
This is **correct by design**. The registry pattern (`get()` returns the class type, caller instantiates) is consistently applied throughout the codebase. No defect, no regression risk, no fix needed.

37. user (2026-06-22T09:49:33.047Z)

Structured output provided successfully

38. assistant (2026-06-22T09:49:37.754Z)

Summary

The finding you asked me to verify is **REFUTED (real=false)** ? this is not a defect.

What the code actually does:
- Line 335 in `nightly_reelcount.py`: `seg_cls = segmenter_registry.get(segmenter)` correctly retrieves the **class type** (not an instance)
- Line 345: `seg_cls(db, config).process_video(...)` correctly **instantiates** the class with the required `Database` and `config` parameters
- This pattern is consistent with the `BasePlaySegmenter.__init__` signature (line 7 in `base.py`), which expects exactly those two arguments

The registry pattern (get returns the type, caller instantiates) is applied consistently throughout the codebase in `backend/main.py`, `backend/worker/__main__.py`, and `tools/generate_highlights.py`. The code has proper null-checks and handles edge cases correctly. No regression risk.